auto_odsx — Automation Scripts Reference

All scripts are Tcl/Expect wrappers around `odsx.py` that automate interactive menu navigation for the GigaSpaces ODSX data engine (Oracle CDC). Run them from any directory; each script `cd`s to `/dbagiga/gs-odsx` internally.

A. Pipeline Operations

1. auto_show_pipeline_interactive

Launches the show-pipeline menu and hands control over to the user at the "Select pipeline" prompt.

```
./auto_show_pipeline_interactive
```

No arguments. Use this when you want to navigate the pipeline details interactively.

---

2. auto_show_pipeline

Shows the details of a specific pipeline (and optionally a specific table) in a fully automated run.

```
./auto_show_pipeline <pipeline_name> [table_name]
```

| Argument | Required | Description | |----------|----------|-------------| | `pipeline_name` | Yes | Exact name of the pipeline as shown in the menu | | `table_name` | No | If omitted, the first table in the pipeline is shown |

Exit codes: `1` if the pipeline or table name is not found.

---

3. auto_create_pipeline_interactive

Launches the create-pipeline menu and hands control over to the user at the "Enter new pipeline name" prompt.

```
./auto_create_pipeline_interactive
```

No arguments. Complete the pipeline name and all subsequent prompts manually.

---

4. auto_import_pipeline

Imports a pipeline from a YAML file that already exists in the pipeline import folder, or imports all pipelines at once.

```
./auto_import_pipeline <pipeline_name|all>
```

| Argument | Description | |----------|-------------| | `pipeline_name` | Name with or without `.yaml` extension (e.g. `my_pipeline` or `my_pipeline.yaml`) | | `all` | Sends `all` to the prompt to import every available pipeline file |

Exit codes: `1` if the file is not found in the import folder.

---

5. auto_export_pipeline

Exports a pipeline to a YAML file, or exports all pipelines at once.

```
./auto_export_pipeline <pipeline_name|all>
```

| Argument | Description | |----------|-------------| | `pipeline_name` | Exact name of the pipeline as shown in the menu | | `all` | Exports every pipeline |

Exit codes: `1` if the pipeline name is not found.

---

6. auto_start_pipeline

Starts a specific pipeline or all pipelines.

```
./auto_start_pipeline <pipeline_name|all>
```

| Argument | Description | |----------|-------------| | `pipeline_name` | Exact name of the pipeline to start | | `all` | Starts every pipeline |

Exit codes: `1` if the pipeline name is not found.

---

7. auto_stop_pipeline

Stops a specific pipeline or all pipelines.

```
./auto_stop_pipeline <pipeline_name|all>
```

| Argument | Description | |----------|-------------| | `pipeline_name` | Exact name of the pipeline to stop | | `all` | Stops every pipeline |

Exit codes: `1` if the pipeline name is not found.

---

8. auto_delete_pipeline

Deletes a specific pipeline or all pipelines. Automatically confirms the destructive "Continue? (yes/no)" prompt.

```
./auto_delete_pipeline <pipeline_name|all>
```

| Argument | Description | |----------|-------------| | `pipeline_name` | Exact name of the pipeline to delete | | `all` | Deletes every pipeline (auto-confirms) |

Exit codes: `1` if the pipeline name is not found.

> **Warning:** This operation is irreversible. The script answers `yes` to the confirmation automatically.

---

9. auto_compare_record_count_pipeline

Runs the compare-record-count operation for a specific pipeline and waits for it to complete.

```
./auto_compare_record_count_pipeline <pipeline_name>
```

| Argument | Description | |----------|-------------| | `pipeline_name` | Exact name of the pipeline to compare |

Exit codes: `1` if the pipeline name is not found.

---

10. auto_compare_index_pipeline

Runs the compare-index operation for a specific pipeline and waits for it to complete.

```
./auto_compare_index_pipeline <pipeline_name>
```

| Argument | Description | |----------|-------------| | `pipeline_name` | Exact name of the pipeline to compare indexes |

Exit codes: `1` if the pipeline name is not found.

B. Schema Structure Management

11. auto_pipeline_add_table_interactive

Launches the add-table menu and hands control over to the user at the "Select pipeline" prompt.

```
./auto_pipeline_add_table_interactive
```

No arguments. Select the pipeline and complete all table configuration prompts manually.

---

12. auto_pipeline_remove_table

Removes a specific table from a specific pipeline. Automatically confirms the "Continue? (yes/no)" prompt.

```
./auto_pipeline_remove_table <pipeline_name> <table_name>
```

Argument	Required	Description
`pipeline_name`	Yes	Exact name of the pipeline
`table_name`	Yes	Space type name or source table name as shown in the pipeline table list

Exit codes: `1` if the pipeline or table name is not found.

> **Warning:** This operation is irreversible. The script answers `yes` to the confirmation automatically.

---

13. auto_pipeline_add_new_column_interactive

Launches the add-new-column menu and hands control over to the user at the "Select pipeline" prompt.

```
./auto_pipeline_add_new_column_interactive
```

No arguments. Select the pipeline, table, and column details manually.

---

14. auto_pipeline_remove_column_interactive

Launches the remove-column menu and hands control over to the user at the "Select pipeline" prompt.

```
./auto_pipeline_remove_column_interactive
```

No arguments. Select the pipeline, table, and column to remove manually.

C. Datasource Operations

15. auto_show_datasource

Lists all configured datasources and exits.

```
./auto_show_datasource
```

No arguments. Output is printed to stdout.

---

16. auto_export_datasource

Exports a specific datasource configuration to a JSON file.

```
./auto_export_datasource <datasource_name>
```

| Argument | Description | |----------|-------------| | `datasource_name` | Exact name of the datasource as shown in the menu |

Exit codes: `1` if the datasource name is not found.

---

17. auto_import_datasource

Imports a datasource from a JSON file that already exists in the datasource import folder.

```
./auto_import_datasource <datasource_name>
```

| Argument | Description | |----------|-------------| | `datasource_name` | Name with or without `.json` extension (e.g. `my_ds` or `my_ds.json`) |

Exit codes: `1` if the file is not found in the import folder.

---

18. auto_test_connection_datasource

Tests the connection for a specific datasource and schema. Automatically selects table number `1` for the connection test.

```
./auto_test_connection_datasource <datasource_name> <schema_name>
```

| Argument | Required | Description | |----------|----------|-------------| | `datasource_name` | Yes | Exact datasource name (with or without `.json` extension) | | `schema_name` | Yes | Exact schema name as shown in the schema selection menu |

Exit codes: `1` if the datasource or schema name is not found.

Notes

- All scripts require the `expect` interpreter (`/usr/bin/expect`).
- Scripts must be run on the host where `/dbagiga/gs-odsx/odsx.py` is accessible.
- Name matching is **exact** (case-sensitive) against the menu table values.
- Scripts ending in `_interactive` launch the menu and pass control to the user; all others are fully automated.

D. Using odsx.py Directly

Every `auto_odsx` script is a wrapper around `odsx.py`. You can invoke the same operations manually either via the interactive menu or by passing the menu path as CLI arguments.

Working directory: Always run `odsx.py` from `/dbagiga/gs-odsx`.

```
cd /dbagiga/gs-odsx
```

Interactive (menu-driven):

```
./odsx.py
```

Navigate: `Data Engine → Oracle-CDC → ...`

CLI (non-interactive): Pass the menu path as space-separated arguments:

```
./odsx.py <menu-level-1> <menu-level-2> ... <operation>
```

D.1 Pipeline Operations

1. Show Pipeline

Displays the full configuration and table details of a selected pipeline, including all mapped tables, replication status, and column-level mappings.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Pipeline → Pipeline-Operations → Show-Pipeline` | | CLI command | `./odsx.py dataengine oracle-cdc pipeline pipeline-operations show-pipeline` |

---

2. Create Pipeline

Creates a new Oracle CDC pipeline by prompting for a pipeline name, datasource, schema, and the set of tables to replicate into GigaSpaces.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Pipeline → Pipeline-Operations → Create-Pipeline` | | CLI command | `./odsx.py dataengine oracle-cdc pipeline pipeline-operations create-pipeline` |

---

3. Import Pipeline

Imports a pipeline configuration from an existing `.yaml` file stored in the pipeline folder, restoring all table mappings and settings.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Pipeline → Pipeline-Operations → Import-Pipeline` | |

CLI command | `./odsx.py dataengine oracle-cdc pipeline pipeline-operations import-pipeline` |

---

4. Export Pipeline

Exports the current pipeline configuration to a `.yaml` file for backup, version control, or migration to another environment.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Pipeline → Pipeline-Operations → Export-Pipeline` | | CLI command | `./odsx.py dataengine oracle-cdc pipeline pipeline-operations export-pipeline` |

---

5. Start Pipeline

Starts CDC replication for a selected pipeline, initiating continuous data capture from the Oracle source and streaming changes into GigaSpaces.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Pipeline → Pipeline-Operations → Start-Pipeline` | | CLI command | `./odsx.py dataengine oracle-cdc pipeline pipeline-operations start-pipeline` |

---

6. Stop Pipeline

Stops CDC replication for a running pipeline, halting data capture without deleting the pipeline configuration (can be restarted).

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Pipeline → Pipeline-Operations → Stop-Pipeline` | | CLI command | `./odsx.py dataengine oracle-cdc pipeline pipeline-operations stop-pipeline` |

---

7. Delete Pipeline

Permanently removes a pipeline and all its associated configuration from the system. This action cannot be undone.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Pipeline → Pipeline-Operations → Delete-Pipeline` | | CLI command | `./odsx.py dataengine oracle-cdc pipeline pipeline-operations delete-pipeline` |

> **Warning:** The delete operation will prompt `Continue? (yes/no)` — confirm carefully.

---

8. Compare Record Count Pipeline

Compares the row counts between Oracle source tables and the corresponding GigaSpaces space objects for a selected pipeline, reporting any discrepancies to validate data completeness.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Pipeline → Pipeline-Operations → Compare-Record-Count-Pipeline` | | CLI command | `./odsx.py dataengine oracle-cdc pipeline pipeline-operations compare-record-count-pipeline` |

---

9. Compare Index Pipeline

Compares index definitions between the Oracle source schema and the target GigaSpaces space for a selected pipeline, detecting any index drift or missing indexes.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Pipeline → Pipeline-Operations → Compare-Index-Pipeline` | | CLI command | `./odsx.py dataengine oracle-cdc pipeline pipeline-operations compare-index-pipeline` |

D.2 Schema Structure Management

10. Add New Column

Enables replication of one or more columns that were previously excluded from a pipeline table mapping. Internally, columns are excluded via an `excludeFields` list in the pipeline YAML — this operation removes selected fields from that list so they start being replicated.

Internal steps performed automatically:

1. Connects to the DI Manager (MDM node) and authenticates via `dihctl`.
2. Lists all pipelines — blocked if the selected pipeline is in `ERROR` state.
3. Exports the pipeline's current YAML configuration.
4. Shows the table pipelines (space types) in the selected pipeline; user picks one.
5. Displays the current `excludeFields` list for that space type (columns currently NOT replicated).
6. User selects which excluded fields to re-include (supports single, range `1-3`, or list `1,4,5`).
7. Confirms intent — warns that the pipeline will be **stopped, deleted, and recreated**.
8. Removes selected fields from `excludeFields` and saves the updated YAML.
9. Stops the pipeline via REST API and polls every 10 s until status is `INACTIVE`.
10. Deletes the pipeline and validates deletion (up to 5 retries).
11. Unregisters the affected space type from Object Management and verifies its removal.
12. Reimports the updated YAML (`dihctl apply`) to recreate the pipeline with the new column included.

> **Note:** The pipeline is temporarily stopped during this operation. Do not run while a critical replication window is active.

Mode	Command / Path
Menu path	`Data Engine → Oracle-CDC → Pipeline → Schema-Structure-Management → Add-New-Column`
CLI command	`./odsx.py dataengine oracle-cdc pipeline schema-structure-management add-new-column`

---

11. Remove Column

Stops replication of one or more columns from a pipeline table by adding them to the `excludeFields` list in the pipeline YAML. The column data already in the GigaSpaces space is not deleted, but no further changes from Oracle will be captured for those columns.

Internal steps performed automatically:

1. Connects to the DI Manager (MDM node) and authenticates via `dihctl`.
2. Lists all pipelines — blocked if the selected pipeline is in `ERROR` state.
3. Exports the pipeline's current YAML configuration.
4. Shows the table pipelines (space types); user picks one.
5. Queries the Object Management API (`/list`) to retrieve the registered columns for the selected space type, showing: column name, data type, space ID flag, routing flag, index type, and tier criteria.
6. User selects which columns to exclude (supports single, range `1-3`, or list `1,4,5`).
7. Confirms intent — warns that the pipeline will be **stopped, deleted, and recreated**.
8. Adds selected columns to `excludeFields` in the YAML and saves the updated file.
9. Stops the pipeline via REST API and polls every 10 s until status is `INACTIVE`.
10. Deletes the pipeline and validates deletion (up to 5 retries).
11. Unregisters the affected space type from Object Management.
12. Reimports the updated YAML (`dihctl apply`) to recreate the pipeline without those columns.

> **Note:** The pipeline is temporarily stopped during this operation.

Mode	Command / Path
Menu path	`Data Engine → Oracle-CDC → Pipeline → Schema-Structure-Management → Remove-Column`
CLI command	`./odsx.py dataengine oracle-cdc pipeline schema-structure-management remove-column`

---

12. Add Table

Extends an existing pipeline by adding a new Oracle source table as an additional table pipeline (space type), enabling CDC replication for that table into GigaSpaces.

Internal steps performed automatically:

1. Connects to the DI Manager (MDM node) and authenticates via `dihctl`.
2. Lists all existing pipelines; user selects the pipeline to extend.
3. Exports the pipeline's current YAML configuration.
4. Connects to the Oracle datasource (via MDM) and lists available schemas; user selects a schema.

5. Lists tables within the selected schema; user selects the table to add.
6. Lists available columns for that table; user selects which columns to include (or exclude).
7. Prompts for space type name and routing/index configuration for the new table.
8. Updates the pipeline YAML with the new `tablePipeline` entry.
9. Stops the pipeline via REST API and polls until `INACTIVE`.
10. Deletes the existing pipeline, validates deletion (up to 5 retries).
11. Reimports the updated YAML (`dihctl apply`) to recreate the pipeline with the new table included.
12. Registers the new space type in Object Management.

> **Note:** The pipeline is temporarily stopped during this operation.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Pipeline → Schema-Structure-Management → Add-Table` | | CLI command | `./odsx.py dataengine oracle-cdc pipeline schema-structure-management add-table` |

---

13. Remove Table

Removes one table pipeline (space type) from an existing pipeline, permanently stopping CDC replication for that Oracle source table. Unlike Add/Remove Column, this operation uses the DIH REST API directly rather than YAML manipulation — the table pipeline record is deleted via `DELETE /api/v2/pipeline/{id}/tablepipeline/{tp_id}`.

Internal steps performed automatically:

1. Connects to the DI Manager (MDM node) and authenticates via `dihctl`.
2. Lists all pipelines — blocked if the selected pipeline is in `ERROR` state.
3. Fetches the pipeline ID via the v2 REST API (`/api/v2/pipeline/`).
4. Fetches the list of table pipelines via port 6081 (`/api/v1/pipeline/{id}/tablepipelines`), showing: space type name, source table name, and table pipeline ID.
5. **Prevents removal if only one table pipeline remains** — a pipeline must have at least one table.
6. User selects which table to remove.
7. Confirms intent — warns that the pipeline will be **stopped** and the table permanently removed.
8. Stops the pipeline via REST API and polls every 10 s until status is `INACTIVE`.
9. Calls `DELETE /api/v2/pipeline/{pipeline_id}/tablepipeline/{tp_id}` to remove the table pipeline record.

> **Warning:** This operation is irreversible. The table pipeline is deleted via API and cannot be restored without manually re-adding the table. The space type data already in GigaSpaces is **not** automatically deleted — clean up separately if needed.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Pipeline → Schema-Structure-Management →`

`Remove-Table` | | CLI command | `./odsx.py dataengine oracle-cdc pipeline schema-structure-management remove-table` |

D.3 Datasource Operations

14. Show Datasources

Lists all configured Oracle CDC datasources with their connection details, including host, port, service name, and username.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Datasource → Show-Datasources` | | CLI command | `./odsx.py dataengine oracle-cdc datasource show-datasources` |

---

15. Import Datasource

Imports a datasource configuration from a `.json` file stored in the datasource folder, registering the Oracle connection details into the system.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Datasource → Import-Datasource` | | CLI command | `./odsx.py dataengine oracle-cdc datasource import-datasource` |

---

16. Export Datasource

Exports a datasource configuration to a `.json` file for backup, documentation, or migration to another ODSX environment.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Datasource → Export-Datasource` | | CLI command | `./odsx.py dataengine oracle-cdc datasource export-datasource` |

---

17. Test Datasource Connection

Tests Oracle database connectivity for a datasource by selecting a schema and running a live connection probe against a table, confirming credentials and network reachability.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Datasource → Test-Datasource-Connection` | | CLI command | `./odsx.py dataengine oracle-cdc datasource test-datasource-connection` |

D.4 Schema Change

18. Oracle-CDC Schema Change

Handles DDL schema changes (e.g. added/dropped/renamed columns) propagated from the Oracle source, updating the pipeline mappings and space type definitions to reflect the new structure.

| Mode | Command / Path | |---|---| | Menu path | `Data Engine → Oracle-CDC → Schema-Change` | | CLI command | `./odsx.py dataengine oracle-cdc schema-change` |

---

odsx.py Menu Hierarchy Reference

```
odsx.py
└── Data Engine
    └── Oracle-CDC
        ├── Schema-Change
        ├── Pipeline
        │   ├── Pipeline-Operations
        │   │   ├── Create-Pipeline
        │   │   ├── Show-Pipeline
        │   │   ├── Start-Pipeline
        │   │   ├── Stop-Pipeline
        │   │   ├── Compare-Record-Count-Pipeline
        │   │   ├── Compare-Index-Pipeline
        │   │   ├── Export-Pipeline
        │   │   ├── Import-Pipeline
        │   │   └── Delete-Pipeline
        │   └── Schema-Structure-Management
        │       ├── Add-New-Column
        │       ├── Remove-Column
        │       ├── Add-Table
        │       └── Remove-Table
        └── Datasource
            ├── Show-Datasources
            ├── Import-Datasource
            ├── Export-Datasource
            └── Test-Datasource-Connection
```